

TrialCheck

Interview Defense Document

v0.2.0 · Platform-agnostic A/B Experiment Readout Auditor

Project	TrialCheck — A/B experiment readout audit library
Version	0.2.0
Language	Python 3.10 / 3.11 / 3.12
Dependencies	Zero — stdlib only (math, statistics, dataclasses)
Tests	17 unit tests across 3 test classes
Checks	7: SRM, primary metric z-test, continuous metric Welch t-test, practical significance, MDE context, peeking risk, guardrail movement, pre-period balance
Demo scenarios	4 canonical: clean_pass, srm_fail, peeking_warn, guardrail_harm
CI	GitHub Actions — matrix Python 3.10/3.11/3.12
Claim boundary	Audit helper. Does not run experiments, make ship/no-ship decisions, implement sequential testing, or perform CUPED.

0. What TrialCheck Is

TrialCheck audits completed A/B experiment readouts. It accepts a structured summary of an experiment (assignment counts, primary metric conversions or means, optional guardrails and pre-period covariates) and returns a structured PASS / WARN / FAIL report.

The core motivation: a p-value from the primary metric is not sufficient to determine whether an experiment result is trustworthy. Teams routinely ship experiments without checking SRM, peeking risk, guardrail movement, or pre-period balance. TrialCheck packages those checks into a single, auditable library call.

0.1 What TrialCheck Does Not Do

- Run experiments or assign users to variants
- Replace the source experimentation platform's statistical engine
- Implement sequential testing (e.g. mSPRT, always-valid inference)
- Perform CUPED / variance reduction
- Make automatic ship/no-ship decisions
- Prove causal validity of the experiment
- Compute prospective power or required sample sizes

0.2 The Intended Workflow

A data scientist or analytics lead runs TrialCheck at readout time. The report surfaces risks. The human makes the ship decision. TrialCheck is decision support, not a decision-maker.

1. Statistical Implementations

1.1 Sample Ratio Mismatch (SRM)

SRM is detected using a chi-square test with one degree of freedom. For a two-arm experiment with observed counts n_c and n_t and expected control ratio r :

```
expected_control = (n_c + n_t) * r expected_treatment = (n_c + n_t) * (1 - r) chi2 =  
(n_c - expected_c)^2 / expected_c + (n_t - expected_t)^2 / expected_t p_value =  
erfc(sqrt(chi2 / 2)) # exact for df=1
```

The survival function $P(\text{Chi-squared}(1) \geq x) = \text{erfc}(\sqrt{x/2})$ is exact — not an approximation — because a chi-squared variable with $df=1$ is the square of a standard normal. The erfc identity follows directly from this relationship.

- Status: FAIL if $p < \alpha$ (default 0.05)
- A failed SRM means assignment, logging, exposure filtering, or data extraction should be investigated before any metric result is interpreted
- No continuity correction applied — chi-square without continuity correction is the standard for SRM detection at this sample size range

1.2 Primary Metric: Two-Proportion Z-Test

For binary conversion metrics, TrialCheck computes a two-proportion z-test using the pooled proportion under H_0 :

```
p_pool = (successes_c + successes_t) / (n_c + n_t) se = sqrt(p_pool * (1 - p_pool) *  
(1/n_c + 1/n_t)) z = (rate_t - rate_c) / se p_value = erfc(|z| / sqrt(2)) # two-sided
```

- Pooled proportion is the correct SE under H_0 (null is that both arms have equal rates)
- Separate (unpooled) SEs would be used for confidence intervals, not hypothesis testing
- Returns INSUFFICIENT_INPUT if conversions are not provided — no fallback assumptions
- Handles edge cases: $se=0$ (100% or 0% conversion in both arms) returns None, None

1.3 Continuous Metric: Welch's T-Test

When VariantSummary.mean and VariantSummary.std are supplied, TrialCheck runs Welch's two-sample t-test rather than Student's t-test:

```
t = (mean_t - mean_c) / sqrt(s_c^2/n_c + s_t^2/n_t) # Welch-Satterthwaite degrees of  
freedom: dof = (s_c^2/n_c + s_t^2/n_t)^2 / ((s_c^2/n_c)^2/(n_c-1) +  
(s_t^2/n_t)^2/(n_t-1))
```

Welch's t-test (not Student's) is the correct default for A/B experiments because:

- The treatment may change variance (e.g. a high-value feature attracts higher-spending users with higher revenue variance)

- Assuming equal variances when they differ inflates Type I error
- Welch's test is slightly conservative when variances are equal, never anti-conservative
- This is consistent with Welch (1947) and is the default in R's `t.test()` and SciPy's `ttest_ind`

P-value computation: Python 3.12+ uses `math.betainc` for an exact regularized incomplete beta function evaluation: $P = I(\text{dof}/(\text{dof}+t^2), \text{dof}/2, 0.5)$. Python 3.10/3.11 fall back to the normal approximation via `erfc` — valid when $\text{dof} > 30$, which holds whenever both arms have $n > 31$. The fallback is explicitly documented in code and in `METHODOLOGY.md`.

The check returns `INSUFFICIENT_INPUT` and is skipped when mean/std are absent, so binary conversion experiments are unaffected.

1.4 Practical Significance

`TrialCheck` compares the observed absolute lift to a user-supplied business threshold. Status is `WARN` (not `FAIL`) when lift is below threshold — a statistically significant but practically tiny result is not a hard block, but it warrants caution before shipping.

- Threshold is absolute lift, not relative — avoids the denominator problem with low base rates where 5% relative lift is 0.0005 absolute
- Returns `INSUFFICIENT_INPUT` if threshold is not provided (caller must declare what 'meaningful' means for their metric)

1.5 MDE Context

Observed absolute lift is compared against the planned minimum detectable effect from the experiment plan. If observed lift $<$ MDE, the result may be weaker than what the experiment was designed to detect.

Important framing: This is context, not a retrospective power calculation. Post-hoc power is statistically controversial (Hoenig & Heisey 2001) because it is algebraically equivalent to a function of the p-value and adds no new information. `TrialCheck` avoids post-hoc power entirely. The MDE check simply asks: 'is the observed effect at least as large as what we planned to be able to detect?'

1.6 Peeking / Early Readout Risk

`TrialCheck` flags peeking risk when: (a) actual duration is less than 70% of planned duration, or (b) `interim_looks > 0` is reported. This is a heuristic, not formal sequential testing. The 70% threshold is a conservative rule of thumb.

- This does NOT implement sequential testing (mSPRT, always-valid p-values, alpha spending)
- The correct fix for peeking is a proper sequential design, not post-hoc adjustment
- The check is honest about its limitations: it raises `WARN`, not `FAIL`

1.7 Guardrail Movement

Each guardrail is declared with a harmful direction ('increase' or 'decrease') and a tolerance. If movement in the harmful direction exceeds tolerance, the check returns FAIL. Within tolerance returns PASS. Status is FAIL (not WARN) because a guardrail breach is typically a hard block on shipping.

1.8 Pre-Period Balance (SMD)

For each pre-period covariate, TrialCheck computes the standardized mean difference:

$$\text{SMD} = (\text{treatment_mean} - \text{control_mean}) / \sqrt{(\text{sd_c}^2 + \text{sd_t}^2) / 2}$$

- Pooled SD (average of both variances, not weighted by n) is Cohen's d formulation — appropriate for equal or near-equal group sizes
- Default threshold: $|\text{SMD}| > 0.10$ triggers WARN — conventional threshold in experiment literature for meaningful pre-period imbalance
- Returns WARN, not FAIL — imbalance is concerning but not always fatal; regression adjustment or stratification may be appropriate remedies

2. Design Decisions

2.1 Zero Dependencies

All statistical computations use Python's standard library only. No numpy, scipy, or pandas. Reasons:

- Installable in any Python environment without version conflicts
- Runnable in interview/demo environments where package installation is restricted
- Forces explicit, auditable computation — no hidden scipy internals
- The math used (chi-square via `erfc`, z-test via pooled proportion, Welch t via Satterthwaite) has closed-form implementations in `stdlib`

2.2 `INSUFFICIENT_INPUT` as a First-Class Status

Most audit tools either fail silently (skip a check without warning) or raise exceptions when inputs are missing. TrialCheck uses a fourth status `INSUFFICIENT_INPUT` that is explicit, visible in the report, and carries a recommendation.

This means a partial readout (e.g. no guardrails provided) is auditable: the report shows which checks were skipped and why. The overall status rolls up `INSUFFICIENT_INPUT` as `WARN`, signaling an incomplete audit rather than a clean pass.

2.3 Separation of Audit from Platform

TrialCheck accepts summary statistics (`n`, conversions, mean, std) rather than raw unit-level data. This is deliberate: in practice, experiment readouts are often reviewed from summary exports (warehouse tables, platform dashboards, spreadsheets). The audit should be runnable from whatever summary the team has.

2.4 `WARN` vs `FAIL` Semantics

The distinction between `WARN` and `FAIL` is intentional:

- `FAIL` = hard readout risk that should block a ship decision (SRM, guardrail breach)
- `WARN` = potential risk that requires human judgment (peeking, practical significance, pre-period imbalance, primary metric not significant)
- `INSUFFICIENT_INPUT` = required data not provided; check skipped

This mirrors how senior data scientists actually review readouts — not every concern is equally blocking.

3. Hard Interview Questions

Q: Why not use scipy for the t-test p-value instead of the betainc fallback?

Two reasons. First, scipy is not a dependency and adding it would break the zero-dependency contract. Second, Python 3.12 added `math.betainc` which is an exact implementation. For 3.10/3.11, the normal approximation is valid when $\text{dof} > 30$, which is almost always true for A/B experiments with reasonable sample sizes. The fallback is conservative — it slightly overstates the p-value — so it will occasionally produce false negatives (miss significance) but never false positives. The fallback path is explicitly documented in `METHODOLOGY.md` and in the source code comment.

Q: Why pooled SE for the z-test instead of unpooled?

The two-proportion z-test under H_0 (null hypothesis that both arms have equal rates) should use the pooled proportion to estimate the common rate. If the null is true, pooling gives a more efficient (lower variance) estimate of the SE. Unpooled SE is appropriate for confidence intervals (where you don't assume a common rate) but not for null hypothesis testing. This is standard in every statistics textbook covering proportion tests.

Q: Why Welch's t-test and not Student's?

Student's t-test assumes equal variances across arms. For A/B experiments, this assumption often fails — a treatment that changes the mean frequently also changes the variance. Welch's test does not make this assumption, is slightly conservative when variances are equal, and is never anti-conservative. R's `t.test()` uses Welch by default for the same reason. The cost is a fractional degrees of freedom, which is handled by the Welch-Satterthwaite equation.

Q: Why no continuity correction in the SRM chi-square test?

Yates' continuity correction is sometimes applied to 2x1 or 2x2 chi-square tests to correct for the discrete nature of count data. However, at the sample sizes typical for SRM detection (thousands of users per arm), the correction is negligible and not standard practice. TrialCheck targets a practical use case, not theoretical edge cases with $n < 30$ per arm.

Q: Why is peeking risk a WARN and not a FAIL?

Because the appropriate response to peeking depends on context. If the experiment used a pre-planned sequential design (group sequential, alpha spending, always-valid inference), then an early call is legitimate. TrialCheck does not have access to that design information — it only sees timing metadata. A WARN correctly signals that a conventional p-value may not be valid, without falsely blocking experiments that were designed for early stopping.

Q: Why absolute lift instead of relative lift for practical significance?

Relative lift creates denominator problems. A 5% relative improvement on a 0.1% conversion rate is 0.005 absolute — almost certainly not worth shipping. Absolute lift forces the caller to declare the minimum movement that is meaningful in their specific metric and context. This is the correct framing for a business threshold.

Q: What happens with extreme conversion rates (near 0% or 100%)?

If both arms have 0% conversion (or both have 100%), the pooled SE is zero and the z-test returns (None, None). The primary metric check returns INSUFFICIENT_INPUT with a recommendation to verify inputs. This is the correct behavior — dividing by zero to produce a spurious z-statistic would be worse than returning None.

Q: Is the SMD threshold of 0.10 defensible?

The 0.10 threshold is the default and is configurable per covariate via `max_abs_smd` in `PrePeriodCovariate`. The 0.10 value is conventional in the experimentation literature for 'no meaningful imbalance', consistent with guidance from Stuart et al. (2008) on balance in observational studies. For high-stakes experiments or known confounders, the caller should tighten the threshold.

4. Claim Boundary

TrialCheck is transparent about what it does not do:

- Does not run experiments or assign users
- Does not replace the source experimentation platform's statistical engine
- Does not implement sequential testing — conventional p-values are not valid after peeking, and TrialCheck does not correct for this
- Does not implement CUPED or any variance reduction technique
- Does not compute prospective power or required sample sizes
- Does not make automatic ship/no-ship decisions
- Does not prove causal validity

The post-hoc MDE context check is explicitly framed as context (observed vs planned MDE) and not as a retrospective power calculation. Post-hoc power is algebraically equivalent to a transformation of the p-value and has been criticized as uninformative (Hoenig & Heisey 2001, American Statistician).

5. Resume-Safe Claim

Built TrialCheck, a platform-agnostic Python library (zero dependencies, 17 tests, CI on Python 3.10/3.11/3.12) that audits completed A/B experiment readouts against seven checks: sample-ratio mismatch (chi-square $df=1$), primary metric two-proportion z-test, continuous metric Welch t-test (Welch-Satterthwaite dof), practical significance, MDE context, peeking risk, guardrail movement, and pre-period covariate balance (SMD). Produces structured JSON/Markdown/HTML reports with explicit PASS/WARN/FAIL/INSUFFICIENT_INPUT status per check, claim boundary, and four canonical demo scenarios covering clean-pass through guardrail-harm readouts.

6. Files Index

src/trialcheck/stats.py	chi_square_df1_survival, two_proportion_z_test, welch_t_test — all statistical primitives
src/trialcheck/checks.py	7 check functions: srm, primary_metric, continuous_metric, practical_significance, mde_context, peeking_risk, guardrail, pre_period_balance
src/trialcheck/auditor.py	TrialCheck class — orchestrates all checks, computes overall status, generates interpretation
src/trialcheck/models.py	VariantSummary, ExperimentSummary, GuardrailMetric, PrePeriodCovariate, CheckResult, TrialReport
src/trialcheck/reporting.py	to_json, to_markdown, to_html, write_report — output rendering
src/trialcheck/io.py	load_experiment_json — CSV/JSON input adapters
src/trialcheck/cli.py	trialcheck-demo CLI entry point
tests/test_trialcheck.py	17 unit tests: TrialCheckTests (6), ContinuousMetricTests (5), EdgeCaseTests (6)
scripts/generate_demo_reports.py	4-scenario demo: clean_pass, srm_fail, peeking_warn, guardrail_harm
METHODOLOGY.md	Statistical methodology including Welch t-test derivation and betainc fallback
CHANGELOG.md	Version history: v0.2.0 (Welch t-test, 4-scenario demo, CI) and v0.1.0
.github/workflows/ci.yml	Matrix CI: Python 3.10 / 3.11 / 3.12
.github/workflows/publish.yml	Trusted publisher OIDC release to PyPI